

Ασκηση 2

Για το πρώτο μέρος της άσκησης δημιουργήθηκαν 3 κλάσεις : η STR.java, Node.java και Point.java. Η Point.java έχει πληροφορίες για ένα σημείο, δηλαδή τον αριθμό της σειράς, το x και το y του, μαζί με μεθόδους getter και setter. Η Node.java έχει το nodeId, n, f και μία λίστα ArrayList<double[]> όπου κρατάει πληροφορίες για τον κόμβο, δηλαδή κάθε πίνακας double[] έχει το node-id ή record-id και τις συντεταγμένες του σημείου ή του MBR, αναλόγως εάν είναι φύλλο ή όχι. Στην STR.java υπάρχουν οι εξής μέθοδοι :

readFile() : Διαβάζει από το αρχείο τα σημεία και τα αποθηκεύει σε έναν πίνακα ArrayList<Point> points τον οποίο και επιστρέφει.

SortByX() : Ταξινομεί ένα πίνακα με βάση την x συντεταγμένη.

sortByY() : Ταξινομεί ένα πίνακα με βάση την y συντεταγμένη.

dividePoints() : Χωρίζει ένα πίνακα σε stripes, ανάλογα την τιμή του r. Επίσης ταξινομεί κάθε stripe με βάση το y.

createLeafNodes() : Προσθέτει στην global ArrayList<Node> tree όπου είναι το δέντρο, τα φύλλα. Συγκεκριμένα κάθε 51 σημεία μπαίνουν σε ένα leafNode εφόσον ένα node έχει χωρητικότητα 1024 bytes, άρα $1024/(8+8+4) = 51,2$.

createFirstInternalNodes() : Δημιουργεί το δεύτερο επίπεδο του δέντρου, δηλαδή το πρώτο επίπεδο εσωτερικών κόμβων. Συγκεκριμένα, κάθε 28 leafNodes μπαίνουν σε ένα internalNode, διότι λόγω της χωρητικότητας $1024/(32+4)=28,45$.

ConstructTree() : Αναδρομικά υλοποιεί το υπόλοιπο δέντρο με την ίδια λογική της createFirstInternalNodes(), μόνο που τώρα τα MBR δημιουργούνται από άλλα MBR και όχι από σημεία. Επιστρέφει την ρίζα του δέντρου.

printTreeInfo() : Εκτυπώνει πληροφορίες του δέντρου στο τερματικό.

printTreeToFile() : Εκτυπώνει το δέντρο σε ένα αρχείο εξόδου.

Main() : Παίρνει το αρχείο εισόδου από την γραμμή του τερματικού και καλώντας τις ανάλογες μεθόδους με την σειρά τους υλοποιεί την τεχνική sort-tiler-recursive (STR) για να διαβάσει τα δεδομένα από το αρχείο και να φτιάξει ένα R-tree.

Για το δεύτερο μέρος της άσκησης δημιουργήθηκε μία έξτρα κλάση `IncrementalBFNN.java` η οποία έχει τις εξής μεθόδους :

`readTreeFromFile()` : Διαβάζει από το αρχείο εξόδου του πρώτου μέρους, το δέντρο και το αποθηκεύει σε μία `ArrayList<Node> treeFromFile`.

`mindist()` : Υπολογίζει την απόσταση μεταξύ ενός σημείου και ενός MBR.

`BNFF()` : Υλοποιεί τον αλγόριθμο best-first nearest neighbor. Αποθηκεύει σε μία `priority queue entries` τύπου `double[]` όπου περιέχουν την απόσταση του από το σημείο που ψάχνουμε, το `nodeid` και τις συντεταγμένες (σημείου ή MBR). Και ακολουθώντας τον αλγόριθμο βρίσκει τους κ πρώτους γείτονες, εκτυπώνοντας και την `priority queue` κάθε φορά που βρίσκει ένα γείτονα.

`findKPlusOneNeighbor()` : Βρίσκει τον αμέσως επόμενο γείτονα, ακολουθώντας την ίδια λογική και δουλεύοντας στην ίδια `priority queue`.

`printNearestNeighbors()` : Εκτυπώνει τους κ γείτονες στο τερματικό.

`Main()` : Παίρνει το σημείο που θέλουμε να βρούμε τους γείτονες, από ποιο αρχείο να διαβάσει το δέντρο και πόσους γείτονες να βρει και καλώντας τις κατάλληλες μεθόδους υλοποιεί τον `incremental nearest neighbor search` αλγόριθμο που βασίζεται σε `best-first search`.